Big-Oh notation

William Hendrix

Lecture 1

Today

- Syllabus
- RAM model of computation
- Big-Oh notation
  - Motivation
  - Proof review
  - Formal definitions

Syllabus

- Read at: sit.instructure.com
- Contact: whendrix@stevens.edu
- Office hours:
  - TR 2:00-3:00 pm
  - GS 251
- CAs: Milan Chandiramani
- Office hours:
  - Milan: TBD

- Course objectives, grading scale, exams, etc.
- Class participation
- Feedback form
- Slides

What will we learn in this class?

- **How to determine if an algorithm is efficient**
  - RAM model
  - Big-Oh definition and properties
- **How to improve your algorithms by organizing data**
  - Stacks and queues
  - Binary search trees
  - Priority queues and heaps
  - Hash tables
- **How to develop your own algorithms**
  - Greedy algorithms
  - Divide-and-conquer
  - Dynamic programming
  - Graph traversals
  - Backtracking
- **Classical sort, search, and graph algorithms**

Why is this class important?

- **Frequent source for interview questions**
  - Along with Data Structures
  - Good practice: LeetCode and HackerRank

- **Algorithm analysis is a useful skill**
  - Understand exactly what Big-Oh notation means
  - Distinguish between efficient and inefficient solutions
  - Appreciate trade-offs between different approaches

- **Caveat:** CS 590 is very rigorous
  - Will likely take more effort than other classes
  - Visit office hours!
  - Read the textbook!
  - Look up YouTube videos on course topics!
  - Solve exercises from textbook or online!

RAM model of computation

- Set of assumptions that make analysis more reasonable

<u>Assumptions</u>

1. All "basic" operations (assignment, arithmetic, branching, memory access, etc.) take 1 operation
   - Loops and functions do not qualify
2. We have "infinite" memory

Cons

- Different operations take different number of clock cycles
  - Cache locality has significant impact
- Virtual memory can slow performance

Pros

- Can actually analyze algorithms

RAM model example

Input: $data$: array of integers
Input: n: size of $data$
Output: index min such that
$data[min] \leq data[j]$, for all j
from 1 to n

```
Algorithm: FindMin
min = 1
for i = 2 to n do
    if data[i] < data[min] then
        min = i
    end
end
return min
```

Big-Oh notation

- Technique for *abstracting away details* of complexity
  - Can be used for time complexity, space complexity, etc.

- **Main idea:** most important aspect of complexity is *how fast it grows* relative to input size
  - Focus on asymptotic (eventual) growth rate
  - "Fast" functions will eventually pass "slow" functions for large *n*
  - Coefficients only matter if growth rate is similar
  - Predicting behavior for small *n* is difficult and often pointless

- Big-Oh notation
  - Organizes growth rates into classes
  - Three main symbols: $O(f(n)), \Omega(f(n)), \Theta(f(n))$
    - Analogous to "at most", "at least", and "similar to" *f(n)*

Justification of Big-Oh

- Algorithm runtime with c=1, running at 1 GHz:

n=	lg(n)	n	$n$lg(n)	n^2	n^3	2^n	n!
10	3 ns	10 ns	33 ns	100 ns	1 µs	1 µs	3.6 ms
20	4 ns	20 ns	86 ns	400 ns	8 µs	1 ms	77 yrs
30	5 ns	30 ns	147 ns	900 ns	27 µs	1 s	
40	5 ns	40 ns	213 ns	1.6 µs	64 µs	18.3 min	
50	6 ns	50 ns	282 ns	2.5 µs	125 µs	13 days	
100	7 ns	100 ns	644 ns	10 µs	1 ms		
1,000	10 ns	1 µs	9.97 µs	1 ms	1 s		
1,000,000	20 ns	1 ms	19.9 ms	16.7 min	31.7 yrs		
1,000,000,000	30 ns	1 s	29.9 s	31.7 yrs			

"Fails" at:	Never!	billions	millions	10k	40ish	16ish

Lesson: on large data, coefficients not as important

The formal definition

- We will use the *formal definition* of Big-Oh to *mathematically prove* claims about Big-Oh
- **<u>Goals</u>**
  - Understand exactly what Big-Oh means
  - Understand the difference between Big-Oh, Big-Theta, and Big-Omega
    - E.g., $O(n^2)$ vs. $\Theta(n^2)$ vs. $\Omega(n^2)$

- We'll start by reviewing mathematical proofs

Mathematical proofs

- **Proof:** convincing mathematical argument for some *claim*
  - Each statement should follow what came before
  - May use theorems, definitions, algebra, etc.

- **Proof by example**
  - Proof technique for existential ("there is") claims
  - To show something exists, just give an example!

- **Example proof structure**
  - Consider *this thing*. Note that *it meets all the conditions of the problem for these reasons (…)*. Therefore, there must exist *something that meets these conditions*, namely *the thing from earlier*.
  - Adapt this structure to your problem

Proof by example example

- Prove that 0 is even.
  - **Definition.** An integer x is **even** if and only if there is some integer k such that $x = 2k$.

- **Formal proof:**
  - Consider the integer $k = 0$.
  - Note that $2k = 2(0) = 0$.
  - Thus, there exists an integer k such that $0 = 2k$, namely $k = 0$.
  - Therefore, 0 is even by the definition of even numbers.

- **Shorter version:**
  - 0 is even because $0 = 2(0)$.
- Still a valid proof <u>as long as</u> the reader can follow what you left out

Direct proof

- Straightforward proof technique
  - Used to prove claims of the form "If A, then B"
- **Main idea:**
  - Assume "if" part is true
  - Argue that the "then" part is true

- **Example proof structure**
  - Suppose that *A is true*. *For these reasons (…)*, we can see that *B must be true*. Therefore, *if A, then B*.
  - May need several sentences for argument

Direct proof example

- If x is even and $x > 2$, x must be composite.
  - **Definition.** An integer x is **composite** iff there exist integers a and b greater than 1 such that $x = ab$.

- **Formal proof:**
  - Suppose that x is even and $x > 2$.
  - Since x is *even*, there must be some integer k such that $x = 2k$.
  - Since $x > 2$ and $x = 2k$, $k > 1$.
  - There must be two integers a and $b > 1$ such that $x = ab$
    - Namely $a = 2$ and $b = k$.
  - Therefore, x must be composite.

Writing a proof

- Proofs are **not** created in the way they are written

1. Proof technique tells you how proof should begin and end
2. Apply relevant definitions
3. Manipulate things you know are true to move towards the goal
4. Work backwards from the goal towards the things you know
   – "How could I conclude this is true?"
5. Repeat 2-4 until the two sides meet
6. (*Optional*) Produce a formal write-up that starts at the beginning and continues to the end

- With practice, you can "see" the chain of logic more quickly

Writing a proof

1. Use proof technique to write start and goal
2. Work forwards and backwards until solved

- **Example:** if x is even and $x > 2$, x is composite.
- Proof:
  - x is even
  - $x > 2$
  - There exists an integer k such that $x = 2k$
  - $2k > 2$
  - $k > 1$
  - $2 > 1$ and $k > 1$
  - There exists integers a and $b > 1$ such that $x = ab$
    - Proof by example: find a and b
    - *Hypothesis:* $a = 2$ and $b = k$
  - x is composite

What we know

Need to show

Existential statements in proofs

- If we know something exists, we can give it a name
- **Example:**
  - There exists an even prime.
  - Let x be that even prime number.
- **Important:** cannot be a variable used earlier in the proof

- **Invalid example:** prove that if x and y are even, $x = y$.
- **Proof:**
  - There exists some integer k such that $x = 2k$,
  - and there exists some integer k such that $y = 2k$.
  - Since $x = 2k$ and $y = 2k$, $x = y$.
- **Correct version:**
  - There exist integers j and k such that $x = 2j$ and $y = 2k$.
    - *Other options:* k_1 and k_2 or k and k'
  - Since we don't know anything else about j and k, we cannot conclude that $x = y$.

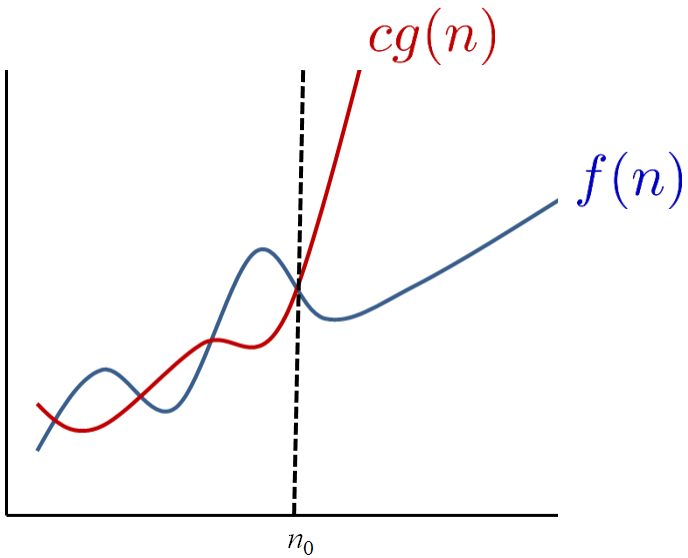
Proof exercise

- Prove that if x and y are even, $x + y$ is even.
  - **Definition.** An integer x is **even** if and only if there is some integer k such that $x = 2k$.

Big-Oh in pictures

- Upper bound *("at most")*

$f(n) = O(g(n))$ if and only if there exist positive constants c and n_0 such that $f(n) \leq cg(n)$ for all $n \geq n_0$.



Translation: f is smaller than some multiple of g <u>eventually</u> (and <u>stays smaller</u>)

Small values of n don't matter

f isn't growing faster than g

Big-Oh

- Upper bound *("at most")*

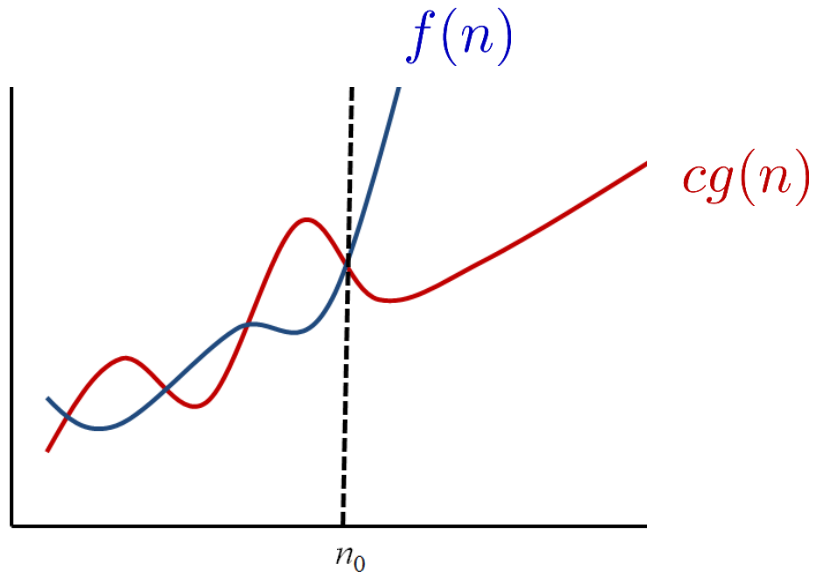
$f(n) = O(g(n))$ if and only if there exist positive constants c and n_0 such that $f(n) \leq cg(n)$ for all $n \geq n_0$.

- We say "*g(n)* dominates *f(n)*" when *f(n) = O(g(n))*
- Notation weirdness:
  - *O*, *Ω*, and *Θ* are classes (sets) of functions
  - BUT: we use = to assign class, not $\in$
- **Example**
  - Prove that $7n^2 + 19n - 4444 = O(n^2)$.

Big-Omega picture

- Lower bound *("at least")*

$f(n) = \Omega(g(n))$ if and only if there exist positive constants c and n_0 such that $f(n) \geq cg(n)$ for all $n \geq n_0$.



Translation: *f* is bigger than some multiple of *g* <u>eventually</u> (and <u>stays bigger</u>)

Small values of *n* don't matter

g isn't growing faster than *f*

Big-Omega

- Lower bound *("at least")*

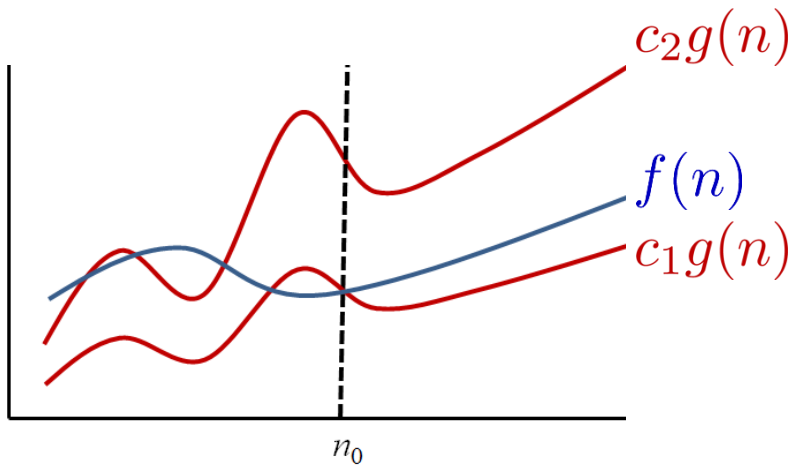
$f(n) = \Omega(g(n))$ if and only if there exist positive constants c and n_0 such that $f(n) \geq cg(n)$ for all $n \geq n_0$.

- **Example**
  - Prove that $7n^2 + 19n - 4444 = \Omega(n^2)$.

Big-Theta picture

- Upper *and* lower bound *("same rate as")*

$f(n) = \Theta(g(n))$ if and only if there exist positive constants c_1, c_2, and n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$.



Translation: *f* can be sandwiched between two multiples of *g* <u>eventually</u> (and <u>stays between them</u>)

f and *g* are growing at the same rate

Small values of *n* don't matter

Big-Theta

- Upper *and* lower bound *("same rate as")*

$f(n) = \Theta(g(n))$ if and only if there exist positive constants c_1, c_2, and n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$.

- **Example**
  - Prove that $7n^2 + 19n - 4444 = \Theta(n^2)$.

Big-Oh exercise

- Use the *formal definition* of Big-Oh to prove:

$$\sum_{i=1}^{n} i = O(n^2)$$

Analysis of Big-Oh

Pros

- Provides a useful summary of the growth rate of the complexity
- Compact
- Simple: eight classes cover most useful algorithms

$$O(1) \ll O(\lg n) \ll O(n) \ll O(n \lg n) \ll O(n^2) \ll O(n^3) \ll O(2^n) \ll O(n!)$$

Cons

- Ignores contributions from coefficients and lower-order terms
- Doesn't rank algorithms with same growth rate
- Doesn't rank algorithms on small inputs
- Some of the "best" algorithms have extremely large coefficients, making them impractical for many purposes

Coming up

- Big-Oh properties
- Algorithm analysis
- Recursive analysis
- Data structures

- **Recommended reading (today):** Sections 1.1 and 1.2
  - *Practice problems:* R-1.3, R-1.7, R-1.19, R-1.21, C-1.8
- **Recommended reading (next week):** Section 1.3
  - *Practice problems:* R-1.11, R-1.15, R-1.26, C-1.3, A-1.4